ABERTURA

Testes de Integração com Spring Boot e JUnit 5

Java 17+ • Spring Boot 3.x • Maven • JUnit 5 • H2

Do teste unitário à integração entre serviço, repositório, banco de dados e HTTP.

Aula prática — Sistema de gerenciamento de produtos

Objetivos da aula

Ao final, os alunos deverão conseguir:

1. Diferenciar testes unitários e testes de integração.
2. Identificar Controller, Service, Repository e banco de dados.
3. Compreender a injeção de dependências.
4. Utilizar @SpringBootTest e JUnit 5.
5. Testar persistência, recuperação e integração HTTP.
6. Compreender o isolamento de dados entre testes.

Recuperando os conceitos anteriores

```
public class EstoqueService {
    private int quantidade;
    public EstoqueService(int quantidade) { this.quantidade = quantidade; }
    public boolean temEstoque(int solicitada) {
        return quantidade >= solicitada;
    }
    public void retirar(int quantidade) { this.quantidade -= quantidade; }
    public int getQuantidade() { return quantidade; }
}
```

O estoque mantém o estado e verifica a disponibilidade.

Integração entre objetos Java

```
public class PedidoService {
    private final EstoqueService estoque;
    public PedidoService(EstoqueService estoque) { this.estoque = estoque; }
    public boolean realizarPedido(int quantidade) {
        if (!estoque.temEstoque(quantidade)) return false;
        estoque.retirar(quantidade);
        return true;
    }
}
```

```
EstoqueService estoque = new EstoqueService(10);
PedidoService pedido = new PedidoService(estoque);
pedido.realizarPedido(3);
```

Pergunta: quantas classes colaboram na operação? Duas.

Teste unitário × teste de integração

Característica	Unitário	Integração
Objetivo	Unidade isolada	Colaboração entre componentes
Dependências	Geralmente isoladas	Componentes reais
Banco de dados	Normalmente substituído	Pode ser utilizado
Velocidade	Geralmente maior	Geralmente menor
Exemplo	Cálculo de desconto	Serviço salva pedido

Um teste de integração não precisa envolver banco de dados.

Aplicação Spring Boot em camadas

Cliente HTTP → ProdutoController → ProdutoService → ProdutoRepository → Banco de dados

Controller: recebe requisições e produz respostas.

Service: implementa regras de negócio.

Repository: realiza operações de persistência.

Banco de dados: armazena registros.

Como verificar que esses componentes funcionam em conjunto?

Injeção de dependências

Em Java convencional, as dependências podem ser criadas manualmente.

```
ProdutoRepository repository = new ProdutoRepository();
ProdutoService service = new ProdutoService(repository);
```

No Spring Boot, o contêiner fornece as dependências dos componentes.

```
@Service
public class ProdutoService {
    private final ProdutoRepository repository;
    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }
}
```

Problema e regras de negócio

Sistema: cadastrar e consultar produtos.

Regra	Comportamento esperado
RN01	Produto deve possuir nome.
RN02	Preço deve ser maior que zero.
RN03	Produto cadastrado deve ser armazenado no banco.
RN04	Produto deve ser recuperado por identificador.

Tecnologias: Java 17+, Spring Boot 3.x, Maven, Spring Data JPA, H2 e JUnit 5.

Entidade Produto

```
package br.edu.exemplo.model;
import jakarta.persistence.*;

@Entity
public class Produto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private double preco;

    protected Produto() {}
    public Produto(String nome, double preco) {
        this.nome = nome; this.preco = preco;
    }
    public Long getId() { return id; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }
}
```

@Entity: entidade persistente. @Id: chave primária. @GeneratedValue: ID gerado.

Repository: persistência com Spring Data JPA

```
package br.edu.exemplo.repository;
import br.edu.exemplo.model.Produto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProdutoRepository
        extends JpaRepository<Produto, Long> {
}
```

Operações fornecidas pelo Spring Data JPA:

save() • findById() • findAll() • deleteById()

Service: validação e cadastro

```
@Service
public class ProdutoService {
    private final ProdutoRepository repository;
    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }
    public Produto cadastrar(String nome, double preco) {
        if (nome == null || nome.isBlank())
            throw new IllegalArgumentException("Nome obrigatório");
        if (preco <= 0)
            throw new IllegalArgumentException("Preço deve ser positivo");
        return repository.save(new Produto(nome, preco));
    }
}
```

Service: busca por identificador

```
public Produto buscar(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException(
            "Produto não encontrado"));
}
```

O que queremos testar?

Recebimento de dados, validações, chamada ao repositório, persistência e recuperação do registro.

@SpringBootTest: contexto da aplicação

```
@SpringBootTest
class ProdutoServiceIntegrationTest {
}
```

Carrega o contexto Spring para os testes.

Atenção:

Não inicia obrigatoriamente um servidor HTTP; isso depende da configuração do teste.

Primeiro teste de integração: preparação

```
@SpringBootTest
@Transactional
class ProdutoServiceIntegrationTest {
    @Autowired
    private ProdutoService service;

    @Autowired
    private ProdutoRepository repository;

    @Test
    void deveCadastrarProdutoNoBanco() {
        String nome = "Teclado";
        double preco = 150.0;
        Produto produto = service.cadastrar(nome, preco);
        // verificações no próximo slide
    }
}
```

Primeiro teste: verificações

```
assertNotNull(produto.getId());
Produto encontrado = repository
    .findById(produto.getId())
    .orElseThrow();
assertEquals("Teclado", encontrado.getNome());
assertEquals(150.0, encontrado.getPreco());
```

Teste → Service.cadastrar() → Repository.save() → H2

Teste → Repository.findById() → H2 → Produto

Verifica a colaboração real entre serviço, repositório e persistência.

Fluxo lógico da persistência

1. Teste chama cadastrar("Teclado", 150).
2. Service valida nome e preço.
3. Repository recebe save(produto).
4. JPA persiste o registro no banco H2.
5. Service devolve produto com identificador.
6. Teste consulta findById(id) e compara os campos.

O momento exato do INSERT/SELECT depende do JPA e do flush().

Regra de negócio: preço inválido

```
@Test
void naoDeveCadastrarProdutoComPrecoNegativo() {
    long quantidadeAntes = repository.count();

    assertThrows(IllegalArgumentException.class,
        () -> service.cadastrar("Mouse", -50));

    assertEquals(quantidadeAntes, repository.count());
}
```

Verifica a rejeição e a ausência de novos registros.

Embora rode em contexto de integração, exercita principalmente a validação do Service.

Isolamento dos testes

Teste A → cadastra produto → banco contém 1 produto

Teste B → espera banco vazio → pode falhar

Os testes não devem depender da ordem de execução.

```
@SpringBootTest
@Transactional
class ProdutoServiceIntegrationTest { ... }
```

Em testes transacionais gerenciados pelo Spring, normalmente ocorre rollback ao final.

Limite: outras threads ou transações independentes podem não participar da transação do teste.

Integração HTTP: ampliando o fluxo

MockMvc → Controller → Service → Repository → Banco H2

MockMvc simula requisições HTTP sem iniciar um servidor HTTP real.

Verificamos também:

Roteamento, desserialização JSON, camada web, integração e resposta HTTP.

Controller: endpoints de produtos

```
@RestController
@RequestMapping("/produtos")
public class ProdutoController {
    private final ProdutoService service;
    public ProdutoController(ProdutoService service) {
        this.service = service;
    }
    @PostMapping
    public Produto cadastrar(@RequestBody ProdutoRequest request) {
        return service.cadastrar(request.nome(), request.preco());
    }
    @GetMapping("/{id}")
    public Produto buscar(@PathVariable Long id) {
        return service.buscar(id);
    }
}
```

DTO de entrada: ProdutoRequest

```
package br.edu.exemplo.controller;

public record ProdutoRequest(String nome, double preco) {}
```

O record representa os dados recebidos no corpo JSON da requisição.

```
{
  "nome": "Teclado",
  "preco": 150.0
}
```

MockMvc: teste de integração HTTP

```
@SpringBootTest
@AutoConfigureMockMvc
@Transactional
class ProdutoControllerIntegrationTest {
    @Autowired private MockMvc mockMvc;
    @Test
    void deveCadastrarProdutoPelaApi() throws Exception {
        mockMvc.perform(post("/produtos")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"nome":"Teclado","preco":150.0}
                """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.nome").value("Teclado"))
            .andExpect(jsonPath("$.preco").value(150.0))
            .andExpect(jsonPath("$.id").isNumber());
    }
}
```

Estratégias de teste no Spring Boot

Estratégia	Objetivo
JUnit 5 sem Spring	Testar classes isoladamente
@DataJpaTest	Testar persistência
@WebMvcTest	Testar camada web isoladamente
@SpringBootTest	Testar integração no contexto completo
@SpringBootTest + MockMvc	Testar fluxos HTTP com componentes reais

Nem todo teste precisa carregar a aplicação inteira.

@DataJpaTest: teste do repositório

```
@DataJpaTest
class ProdutoRepositoryTest {
    @Autowired
    private ProdutoRepository repository;

    @Test
    void devePersistirProduto() {
        Produto produto = new Produto("Mouse", 80);
        Produto salvo = repository.save(produto);
        assertNotNull(salvo.getId());
    }
}
```

Foco na camada de persistência, sem carregar necessariamente todas as camadas da aplicação.

Atividade prática: evolução do cadastro

1. Buscar produto existente: cadastrar e consultar por ID.
2. Buscar produto inexistente: verificar a exceção prevista.
3. Nome vazio: rejeitar e confirmar ausência de persistência.
4. Atualizar preço: cadastrar, alterar e consultar novamente.
5. Remover produto: cadastrar, excluir e verificar ausência.

Três níveis de teste

UNITÁRIO JUnit → ProdutoService → dependência simulada

INTEGRAÇÃO JUnit → ProdutoService → Repository → H2

HTTP MockMvc → Controller → Service → Repository → H2

Os testes se complementam: diferentes níveis verificam riscos diferentes.

Perguntas para discussão

1. Um método correto isoladamente funciona necessariamente integrado? Não: pode haver problemas de configuração, comunicação e persistência.

2. Todo @SpringBootTest é um bom teste de integração? Não: depende das interações efetivamente exercitadas.

3. Por que não usar apenas testes de integração? Testes unitários são geralmente mais rápidos e ajudam a localizar defeitos.

4. Por que não usar apenas testes unitários? Eles não verificam a integração com componentes e infraestrutura reais.

Sequência didática recomendada

Etapa	Conteúdo	Prática
1	Testes unitários	Conta Bancária
2	TDD	Estoque e Carrinho
3	Integração entre objetos	PedidoService + EstoqueService
4	Introdução ao Spring Boot	Controller, Service e Repository
5	Integração com persistência	ProdutoService + Repository + H2
6	Integração HTTP	MockMvc
7	Exercício autônomo	Evolução do cadastro de produtos

Preparação do ambiente

Dependências Maven necessárias:

spring-boot-starter-web

spring-boot-starter-data-jpa

h2

spring-boot-starter-test

Também é necessária a classe principal com @SpringBootApplication.

Os exemplos usam jakarta.persistence e são destinados ao Spring Boot 3.x.

Síntese final

Teste unitário: verifica uma unidade isoladamente.

Teste de integração: verifica a colaboração entre componentes reais.

Integração HTTP: exercita a entrada web e as camadas subsequentes.

Uma suíte de testes consistente combina níveis de teste, regras de negócio e isolamento adequado.